

Module 1: Advanced Android Development

Zero Lecture: Introduction about Advanced Android Development, Syllabus, TLEP , ACTIVITIES Discussion

Lecture 1.Animation: *Animation is the process of creating motion and shape change*

View Animation:

This is the simplest animation used in Android. It define the properties of our Views that should be animated using a technique called Tween Animation. It take the following parameters i.e. size, time duration , rotation angle, start value , end value, and perform the required animation on that object. You can execute the animation by specifying transformations on your View. This can be done in XML resource files or programmatically.

Property Animation:

This animation was introduced in Android 3.0 (API level 11). It allows the user to animate anything.

Property animations are highly customizable, you can specify the duration, the number of repeats, the type of interpolation, and the frame rate of the animation. The Property Animation system is always preferred for more complex animations.

Drawable Animation:

This animation allows the user to load drawable resources and display them one frame after another. This method of animation is useful when user wants to animate things that are easier to represent with Drawable resources.

Graphics: Android provides a range of graphics libraries that allow developers to add images, animations, and other visual elements to their applications. Here are some of the key graphics components in Android:

Canvas: Canvas is a 2D drawing API that allows developers to create custom graphics and animations. It provides a range of drawing methods, such as `drawRect()`, `drawCircle()`, and `drawBitmap()`.

Drawable: Drawable is an abstract class that provides a general framework for displaying graphics and other visual elements.

Animation: Android includes a range of animation classes that allow developers to create animations for their applications.

OpenGL ES: OpenGL ES is a 3D graphics API that allows developers to create advanced 3D graphics and animations.

RenderScript: RenderScript is a high-performance computation engine that allows developers to perform complex image processing and other computations on the device's graphics hardware.

Lecture 2. Work with 2D Graphics: A canvas is the simplest, easiest way to draw 2D objects on the screen. The base class is Canvas.

The Android provides a huge set of 2D-drawing APIs that allow you to create graphics.

There are two ways to create graphics using these API.

- Drawing to a View
- Drawing on a Canvas

If you want to draw graphics that have static graphic content or predefined animation, then draw your graphics or animations into a View object from your layout.

If you want to draw graphics that have dynamic content such as video games, then draw your graphics directly to a Canvas. The Canvas class has several methods for drawing lines, bitmaps, text and many other different types of shapes.

Some of the important methods of canvas class are as follows.

- drawText()
- drawRoundRect()
- drawCircle()
- drawRect()
- drawBitmap()

Create a new java class that should extend from View class. Override the **onDraw()** method. In this method you can use **Canvas class** to draw the different shape.

```
public class MyView extends View
{
    public MyView(Context context)
    {
        super(context);
        // TODO Auto-generated constructor stub
    }
    @Override
    protected void onDraw(Canvas canvas)
    {
        // TODO Auto-generated method stub
        super.onDraw(canvas);
        int radius;
        radius = 50;
        Paint paint = new Paint();
        paint.setStyle(Paint.Style.FILL);
```

```

        paint.setColor(Color.parseColor("#CD5C5C"));
        canvas.drawCircle(150,200, radius, paint);
        canvas.drawRoundRect(newRectF(20,20,100,100), 20, 20, paint);
        canvas.rotate(-45);
        canvas.drawText("Android", 40, 180, paint);
        canvas.restore();
    }
}

```

Following is the content of the modified MainActivity.java file

Note that you have to pass the object of sub class that extends from View class in **setContentview()** method as given below. In our case the name of sub class is MyView.

Public class Main Activity extends Activity

```

{
    @Override
    protectedvoidonCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(newMyView(this));
    }
    @Override
    publicbooleanonCreateOptionsMenu(Menu menu)
    {
        // Inflate the menu; this adds items to the action bar if it is present.
        getMenuInflater().inflate(R.menu.main, menu);
        returntrue;
    }
}

```

Lecture3:

Frame Animation: In Android Frame Animation, you will be swapping frames repeatedly, so that it appears continuous to the human eye and we feel that it is animated. Frame is referred to an image. So to implement frame by frame animation in android, one needs to have set of images, which describes a motion.

1. Identifying the Image Frames

First step towards creating frame animation is to prepare a series of images with the individual frames of your animation. Add the images to your project's drawable folder.

2. Define Animation Sequence

Declare a XML file that defines the animation sequence that holds the list of drawables. Create a new file named `frame_animation_list.xml` and paste it to your project resource `res/anim` or `res/drawable` folder.

```
<?xml version="1.0" encoding="utf-8"?>
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="false">
    <item android:drawable="@drawable/frame1" android:duration="210" />
        <item android:drawable="@drawable/frame2" android:duration="210" />
        <item android:drawable="@drawable/frame3" android:duration="210" />
        <item android:drawable="@drawable/frame4" android:duration="210" />
        <item android:drawable="@drawable/frame5" android:duration="210" />
        <item android:drawable="@drawable/frame6" android:duration="210" />
        <item android:drawable="@drawable/frame7" android:duration="210" />
        <item android:drawable="@drawable/frame8" android:duration="210" />
        <item android:drawable="@drawable/frame9" android:duration="210" />
        <item android:drawable="@drawable/frame10" android:duration="210" />
    </item>
</animation-list>
```

Copy

- Notice that in the above layout declaration, we are using `AnimationDrawable` class. This takes the list of `Drawable` resources and render them at specified intervals.
- Note that the `AnimationDrawable` class, loads all the images into memory before it starts animation.
- You must be careful about using this for images of larger size. Depending on the size of the images, you have to restrict the number of frames for making animation smoother.

3. Declare Activity Layout

Now declare an image view to your activity layout where the animation will be shown. In my example, I have declared an image view and two buttons `Start` and `Stop` to control the animation. Add the following snippet to your `activity_main.xml` file.

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:adjustViewBounds="true" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_alignParentBottom="true"
```

```

        android:layout_marginBottom="20dp"
        android:orientation="horizontal">

        <Button
            android:id="@+id/start"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="0.5"
            android:text="Start" />

        <Button
            android:id="@+id/stop"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="0.5"
            android:text="Stop" />
    </LinearLayout>
</RelativeLayout>

```

Copy

4. Control the Animation Behaviour

Now that we are ready with the animation sequence, we need to add this Drawable as a background resource for our ImageView. Use the following code snippet to set the AnimationDrawable as the background of the ImageView.

```

// Setting animation_list.xml as the background of the image view
view.setBackgroundResource(R.drawable.frame_animation_list);

```

Copy

Once this is set, you can access this AnimationDrawable object by doing a get on the view object like this:

```

// Type casting the Animation drawable
frameAnimation = (AnimationDrawable) view.getBackground();

```

Copy

- Now we can control the behaviour of animation by using `start()` and `stop()` methods to start and stop the animation.
- You may use `setOneShot()` method runs the animation once and then stops.
- The `addFrame(drawable, duration)` method can be used to add a new frame by passing a Drawable object and sets its display duration.

5. Completing Example Code

Now that we understand the concepts, let us complete the example. Here is the complete code that goes inside MainActivity.java.

```

import android.graphics.drawable.AnimationDrawable;
import android.support.v7.app.ActionBarActivity;
import android.os.Bundle;
import android.view.View;

```

```

import android.view.View.OnClickListener;
import android.widget.ImageView;

public class MainActivity extends ActionBarActivity implements OnClickListener {

    private ImageView view;
    private AnimationDrawable frameAnimation;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Type casting the Image View
        view = (ImageView) findViewById(R.id.imageView);

        // Setting animation_list.xml as the background of the image view
        view.setBackgroundResource(R.drawable.frame_animation_list);

        // Type casting the Animation drawable
        frameAnimation = (AnimationDrawable) view.getBackground();

        //set true if you want to animate only once
        frameAnimation.setOneShot(true);

        findViewById(R.id.start).setOnClickListener(this);
        findViewById(R.id.stop).setOnClickListener(this);
    }

    @Override
    public void onClick(View v) {
        int id = v.getId();
        if(id == R.id.start){
            frameAnimation.start();
        }else if(id==R.id.stop){
            frameAnimation.stop();
        }
    }
}

```

Lecture 4: Tween Animation:

This is the simplest animation used in Android. It define the properties of our Views that should be animated using a technique called Tween Animation. It take the following parameters i.e. size, time duration , rotation angle, start value , end value, and perform the required animation on that object. You can execute the animation by specifying transformations on your View. This can be done in XML resource files or programmatically.

Android View animation can make animation on any View objects, such as ImageView, TextView or Button objects. View animation can only animate simple properties like

position, size, rotation, and the alpha property that allows you animate the transparency of a View.

The drawback of this mechanism is that it can only be applied to Views.

Sr.No	Method & Description
1	start() This method starts the animation.
2	setDuration(long duration) This method sets the duration of an animation.
3	getDuration() This method gets the duration which is set by above method
4	end() This method ends the animation.
5	cancel() This method cancels the animation.

Multimedia in Android:

Multimedia in Android refers to the use of audio, video, and images in Android apps. Android provides a range of APIs and tools to handle multimedia, making it easy to incorporate multimedia features into Android apps. Multimedia is an essential aspect of modern Android apps, and it is used in a wide range of applications, from social media to gaming.

Lecture 5: Play Audio Files:

Playing Audio

Android provides several ways to play audio in Android apps. One of the most common ways is to use the MediaPlayer class. The MediaPlayer class provides a way to play audio files, such as MP3 and WAV files. To play audio using the MediaPlayer class, you need to create an instance of the MediaPlayer class and set the audio file to play.

Here is an example of how to play an audio file using the MediaPlayer class:

```
MediaPlayer mediaPlayer = new MediaPlayer();  
mediaPlayer.setDataSource("path/to/audio/file.mp3");  
mediaPlayer.prepare();  
mediaPlayer.start();
```

Recording Audio

Android also provides a way to record audio using the MediaRecorder class. The MediaRecorder class provides a way to record audio from the device's microphone. To record audio using the MediaRecorder class, you need to create an instance of the MediaRecorder class and set the audio source to the device's microphone.

Here is an example of how to record audio using the MediaRecorder class:

```
MediaRecorder mediaRecorder = new MediaRecorder();
```

```
mediaRecorder.setAudioSource(MediaRecorder.AudioSource.MIC);
mediaRecorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
mediaRecorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);
mediaRecorder.setOutputFile("path/to/output/file.3gp");
mediaRecorder.prepare();
mediaRecorder.start();
```

Lecture 6: Play Video Files Work in Background:

Android also provides a way to record video using the MediaRecorder class. The MediaRecorder class provides a way to record video from the device's camera. To record video using the MediaRecorder class, you need to create an instance of the MediaRecorder class and set the video source to the device's camera. Here is an example of how to record video using the MediaRecorder class:

```
MediaRecorder mediaRecorder = new MediaRecorder();
mediaRecorder.setVideoSource(MediaRecorder.VideoSource.CAMERA);
mediaRecorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
mediaRecorder.setVideoEncoder(MediaRecorder.VideoEncoder.H264);
mediaRecorder.setOutputFile("path/to/output/file.3gp");
mediaRecorder.prepare();
mediaRecorder.start();
```

Lecture 7: Services:

Services in Android are a special component that facilitates an application to run in the background in order to perform long-running operation tasks. The prime aim of a service is to ensure that the application remains active in the background so that the user can operate multiple applications at the same time.

1. Foreground Services:

Services that notify the user about its ongoing operations are termed as Foreground Services. Users can interact with the service by the notifications provided about the ongoing task. Such as in downloading a file, the user can keep track of the progress in downloading and can also pause and resume the process.

2. Background Services:

Background services do not require any user intervention. These services do not notify the user about ongoing background tasks and users also cannot access them. The process like schedule syncing of data or storing of data fall under this service.

3. Bound Services:

This type of android service allows the components of the application like activity to bound themselves with it. Bound services perform their task as long as any application component is bound to it.

Notification Services:

Android Notification provides short, timely information about the action happened in the application, even it is not running. The notification displays the icon, title and some amount of the content text.

Some of the notification properties are mention below:

- **setSmallIcon()**: It sets the icon of notification.
- **setContentTitle()**: It is used to set the title of notification.
- **setContentText()**: It is used to set the text message.
- **setAutoCancel()**: It sets the cancellable property of notification.
- **setPriority()**: It sets the priority of notification.

Lecture 8: Broadcast Receiver:

Broadcast in android is the system-wide events that can occur when the device starts, when a message is received on the device or when incoming calls are received, or when a device goes to airplane mode, etc. Broadcast Receivers are used to respond to these system-wide events. Broadcast Receivers allow us to register for the system and application events, and when that event happens, then the register receivers get notified. There are mainly two types of Broadcast Receivers:

- **Static Broadcast Receivers:** These types of Receivers are declared in the manifest file and works even if the app is closed.
- **Dynamic Broadcast Receivers:** These types of receivers work only if the app is active or minimized.
 - Following are some of the important system-wide generated intents:-

Intent	Description Of Event
android.intent.action.BATTERY_LOW :	Indicates low battery condition on the device.
android.intent.action.BOOT_COMPLETED	This is broadcast once after the system has finished booting
android.intent.action.CALL	To perform a call to someone specified by the data
android.intent.action.DATE_CHANGED	Indicates that the date has changed
android.intent.action.REBOOT	Indicates that the device has

	been a reboot
android.net.conn.CONNECTIVITY_CHANGE	The mobile network or wifi connection is changed(or reset)
android.intent.ACTION_AIRPLANE_MODE_CHANGED	This indicates that airplane mode has been switched on or off.

The two main things that we have to do in order to use the broadcast receiver in our application are:

- **Creating the Broadcast Receiver:**

```
class AirplaneModeChangeReceiver:BroadcastReceiver() {
    override fun onReceive(context: Context?, intent: Intent?) {
        // logic of the code needs to be written here
    }
}
```

- **Registering a BroadcastReceiver:**

```
IntentFilter(Intent.ACTION_AIRPLANE_MODE_CHANGED).also {
    // receiver is the broadcast receiver that we have registered
    // and it is the intent filter that we have created
    registerReceiver(receiver,it)
}
```

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:tools="http://schemas.android.com/tools"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    tools:context=".MainActivity">

</androidx.constraintlayout.widget.ConstraintLayout>
```

Step 3: Working with the MainActivity file

Go to the **MainActivity** file and refer to the following code. Below is the code for the **MainActivity** file. Comments are added inside the code to understand the code in more detail.

- Java

Step 4: Create a new class

```
}
```